❶ Replace IConguration with IOptions<SmtpSettings>.

❷ Assign the Value property to the SmtpSettings field.

14.2.5 Binding directly to POCOs

Finally, we will look at binding a configuration section directly to a POCO, using the `IConfiguration` `Bind` method. The hydrated POCO is then registered as a singleton with the DI container and can be injected into any service that requires it. This is possibly the cleanest way to manage strongly typed configuration in that it doesn't depend on `IConfiguration` or `IOptions`.

Listing 14.17 shows the code you use to bind the configuration section directly to an instance of `SmtpSettings`. This replaces the registration of the `SmtpSettings` configuration section you added to Program.cs in listing 14.15.

Listing 14.17 Binding the configuration to a POCO

```
var smtpSettings = new SmtpSettings();
builder.Configuration.GetSection(nameof(SmtpSettings)).Bind(smtpSettings);
builder.Services.AddSingleton(smtpSettings);
```

Within the `ProductionEmailService`, you change the constructor and inject `SmtpSettings` directly, relieving you of having to extract the configuration from `IOptions .Value`:

```
public ProductionEmailService(ILogger<ProductionEmailService> logger,
➥ SmtpSettings settings)
{
    _logger = logger;
    _settings = settings;
}
```